RailFuse ContextStore

面向版本化 KV Cache 对象的故障原子多轨读取引擎

One Worker, many rails, one complete object.

把多条 RDMA 路径读到的字节，变成一个可安全交付的完整对象。

面向 ContextStore 的客户端增量实现：沿用已有磁盘条带与服务端能力，在单个 Worker 内完成路径匹配、按字节分配、并行读取、完整性与版本复查，再统一发布。

本轮强化直达 Python 的原生接口：请求报错返回之前先结束 DMA 所有权，阻止晚到写入污染复用缓冲区；重连前清空旧完成事件。

可运行实现	真实故障对照	可复查证据
Rust 客户端 + C ABI + Python 完整源码与构建入口	旧库精确污染负对照 修复版真实 RXE 恢复	源码与二进制哈希 原始日志、收据与复核

技术版本：39ce2d50fcee00405758f1e2c4f7bd389baccf1c

完整代码与固定提交 验证证据目录

本材料说明技术成果、运行流程、验证范围与应用方向。报名、邮件投递、维护者合并及赛事评审各自独立。

01 从一次对象读取的失败出发

大模型推理可以在 Worker 之间复用已计算的 KV Cache。ContextStore 提供分层存储与共享对象访问；当对象被条带化存储、网络具备多条路径时，客户端需要把这些路径组织成一次正确的对象读取。

挑战	直接并行的风险	本作品的设计
多路径与对象版本	两条路径可能读到不同状态，或只完成部分条带	复用 Placement；传输后复查版本和布局身份
性能与发布安全	直写调用方缓冲区会暴露部分对象；多次组装增加内存搬运	先写私有暂存；可选有边界的直接有序暂存
超时与内存复用	API 已报错，QP 仍可写入；旧 CQE 可能被下一次 PUT 消费	错误前退休 QP；销毁后排空 CQ；明确重连

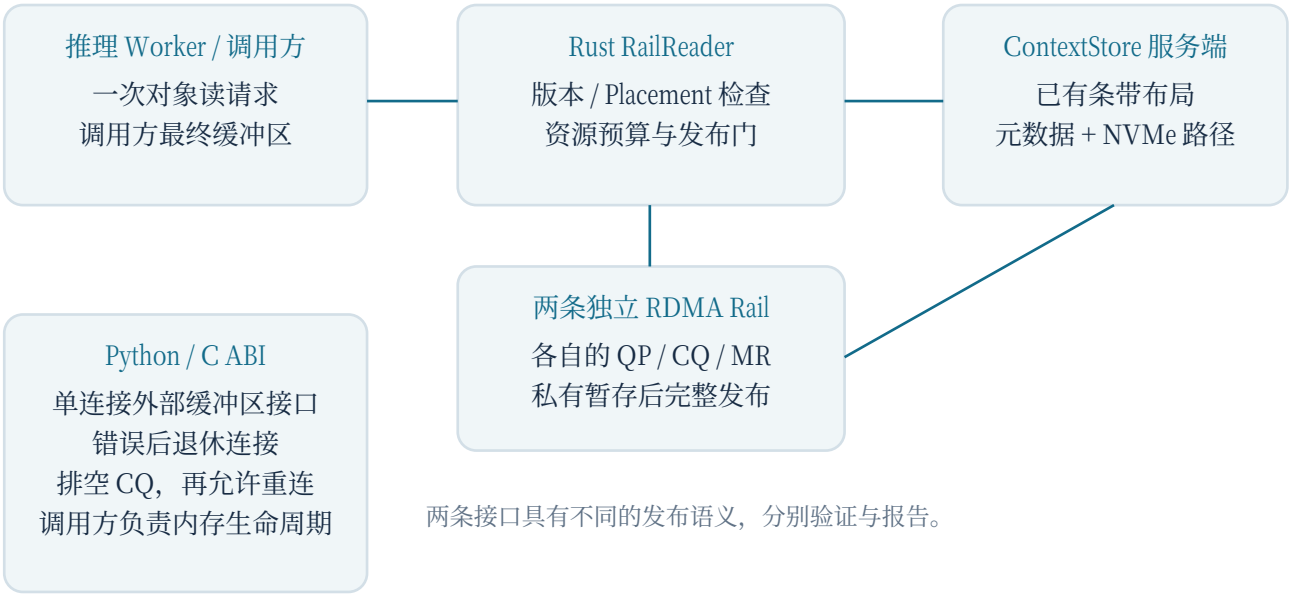
核心问题

当一条路径晚到、对象版本变化或请求报错时，系统是否仍能保证：调用方只接受经校验的完整对象，或者安全地失败；随后复用内存和重连不会继承上次请求的 DMA？

应用收益的定位

已有结果支持系统正确性、真实 Verbs 数据路径与软件开销改进研究。端到端推理吞吐、首 token 时延和业务成本尚需与具体推理框架及真实负载联合测量。

02 架构：保留条带，组织路径与发布



两条接口具有不同的发布语义，分别验证与报告。

Rust 多轨路径

KvClient 查询对象描述符与 Placement，RailReader 匹配本地设备和服务端监听端点，规划每轨条带与资源预算。各轨传输到私有区域，等待所有必要任务结束；完成字节数、校验和及身份复查通过后，才把完整结果交给调用方。

兼容与部署

服务端可发布额外 fabric / listener 能力；客户端也保留显式 route 映射，适配旧服务端。调度只改变读取路径，不重新定义磁盘条带布局。单轨配置仍沿用同一对象 API。

Python / C ABI 路径

现有 Python FFI 是单连接接口，支持调用方注册外部缓冲区。它采用串行生命周期与失败后的 QP 退休，但允许失败 GET 留下已写入的部分内容；它不自动获得 Rust RailReader 的故障原子发布语义。

03 三项相互支撑的增量机制

一、基于对象布局的多轨规划

以对象条带和所属存储端点为输入，匹配可用 Rail 与实际监听地址。按字节规划、保留单轨兼容，并把请求数、注册内存、并发传输和连接数纳入预算。价值是让多路径并行建立在对象语义和资源约束之上。

二、私有暂存与统一发布

传输完成不等于可发布。读取后检查必要字节、校验和、描述符和 Placement 身份；失败、取消或版本变化时保留调用方原值。研究分支提供可选 direct-to-ordered 私有暂存，避免 packed 数据再组装一次，同时保留发布门。

三、跨语言接口的 DMA 生命周期闭环

原生传输出错时先关闭控制连接、销毁 QP并排空 CQ，再把错误交给 Python；清理失败采用 fail-stop。Python 在分配前检查生命周期 ABI，给 native 操作与关闭加锁，并拒绝整数收窄、NUL 字符串别名和无效地址范围。

实现中的取舍

正确性热路径采用显式状态机和资源所有权规则。并行路径与直接暂存的性能效果需要强单轨基线和匹配工作负载；系统不把软件 RXE 的结果解释为物理网卡聚合能力。

04 一个可复现的污染反例

负对照先证明风险真实存在

保留同一注册内存、同一 rkey：GET 收到错误后返回，调用方把缓冲区填成复用哨兵；延迟的真实 GET 随后才到达服务端。旧库未退休 QP，晚到 RDMA WRITE 把哨兵完整改写为服务端 payload。两条 RXE 链路均复现精确内容污染。

观察点	旧库负对照	修复版
请求错误返回后	QP 仍可接受晚到写入	QP 已在返回前退休
缓冲区复用后	哨兵变成精确 payload	哨兵内容及 SHA-256 保持一致
同一客户端句柄恢复	负对照只记录风险	重连后实际 PUT/GET 内容完整

验证顺序不能颠倒

错误返回 → 写入复用哨兵 → 转发延迟请求 → 服务端响应或 EOF → 检查哨兵 → 注销 MR / 重连。不能在晚到请求结束之前提前检查，也不能先换一块缓冲区来宣称复用安全。

精确污染负对照的 payload SHA-256

631b84027d6b9e52b539c4e8373622d23032dfadc64d60af87339c9037e4f769

PUT 的另一条风险是旧完成事件。受控 CQ shim 暂时返回 0，隐藏轮询结果；QP 销毁后读取真实 provider WC，要求它在错误返回前被排空，再用同一句柄、同一注册内存完成三轮 PUT/GET。

05 验证矩阵与原始证据

验证层	本次结果	能支持的结论
Python 回归	165 通过；11 跳过	锁、关闭、参数拒绝、旧库边界及现有功能回归
原生生命周期	13 / 13 通过	真实本地 Verbs 资源与故障注入清理；此层不发数据 WR
真实 RXE 负对照	2 / 2 精确污染	两条链路均出现旧库真实 DMA 污染
真实 RXE 修复版 GET	8 / 8 通过	2 Rail × 2 数据尺寸 × 错误标签 / 超时；同句柄恢复
真实 RXE PUT / CQ	2 / 2 通过	旧真实 WC 在错误前排空；每 Rail 三轮 PUT/GET
构建与代码检查	全量 build、server tests、Clippy、fmt、Black、Ruff 通过	当前源码能构建，所列检查与证据留档

新增的独立复核闭环

审查发现并修复：描述符整数和字符串的静默别名，以及原生 free 已完成、Python 中断造成悬空句柄重试的风险。保留失败回归和修复后结果；描述符只读取一次，再验证并传递同一快照。

新录制的核心流程

当前二进制实际完成 64 MiB 写入与逐字节校验；单轨和双轨读取输出相同对象哈希；双轨各传 32 MiB。该单次运行用于功能演示，不作为稳定性能优势的统计证据。

证据包括每次命令、退出码、JSON、Verbs 事件、完整服务端日志、源码快照和二进制哈希。历史失败和中间轮次也保留。

06 应用场景与使用流程

场景	可使用的能力	还需结合场景验证
共享 KV Cache / 长上下文复用	版本化对象；从存储向单 Worker 多路径读取	推理框架接入、端到端收益及访问局部性
多网络路径的存储节点	既有条带上的 route / listener 映射与按字节规划	物理双 HCA、独立磁盘与瓶颈归因
Python 控制和外部 pinned buffer	原生注册、GET/PUT、串行关闭、错误后重连	调用方持有原始指针和视图的生命周期

从部署到一次读取

1. 在具备 RDMA 依赖的 Linux 环境构建 KVService、Rust SDK 与 FFI；部署元数据、存储目录及 RDMA listener。

2. 配置 Worker 本地设备与 Rail 路由，查询对象版本和 Placement；调用 Rust 多轨对象读取接口。

3. 只消费成功返回的完整对象；发生版本、校验或传输错误时按接口约定处理失败。

4. 使用 Python FFI 时先连接并注册有效缓冲区；传输失败后显式 reconnect，再执行实际 PUT/GET；注销后才释放调用方分配的内存。

演示视频按 “ 环境 → 写入与读取 → 污染反例 → 修复 → 重连 → 复现 ” 展开，使用本轮真实运行记录剪辑与中文配音，并附完整原始输出。

07 复现、开源与交付入口

固定版本与证据

完整代码仓库 / 本次固定提交

提交：39ce2d50fcee00405758f1e2c4f7bd389baccf1c

本轮验证收据与原始日志

四分钟核心功能与安全演示视频

构建与验证入口

项目根目录执行 make build 与 make test-server。Python 回归执行 pytest tests/；原生生命周期使用 run_ffi_lifecycle_validation.py；真实数据面使用 ffi_rxe_late_write.py 与 ffi_rxe_put_reconnect.py。各次启动命令和环境写入证据目录，可据此重放。

软件与硬件前提

Linux、Rust / Cargo、Python（项目要求 3.10+）、libibverbs 与可用 RDMA provider；服务端按配置使用 Redis 和存储目录。本次数据面为两台 KVM 来宾、两条 RXE 网络、同一物理宿主机和虚拟盘。

贡献与证据边界

基于 Apache-2.0 的 DaoCloud ContextStore 开源项目，保留上游协议、服务能力与许可证。独立客户端实现及研究分支的增量见代码与历史；PR #32 的维护者审阅、合并和官方 CI 结果应单独查看。

不把这组 RXE 结果外推为物理双 HCA 带宽聚合、多 NVMe 并行收益或所有交错下的形式化证明。实际服务端精确 commit 时间、物理硬件 provider 故障行为及端到端推理收益仍需专门实测。

按用户提供的通知，作品材料于 2026-10-16 24:00 前发送至 oscc@oschina.cn；本附件准备、代码推送与正式邮件投递分别留存状态。